

PDF 텍스트 매퍼

프로젝트 구조와 데이터 흐름 안내서

화면에서 입력한 값이 어디에 저장되고,
원본 PDF가 어떻게 완성된 PDF로 만들어지는지
한 흐름으로 이해하기 위한 문서

핵심 요약

React 화면에서 Firebase를 직접 사용하며, PDF 생성은 서버가 아니라 사용자의 브라우저에서 처리됩니다.

작성 기준: 2026-07-14 현재 코드

프로젝트를 한 문장으로 보면

하나의 워크스페이스 안에서 표 데이터를 입력하고, 원본 PDF에 출력 위치를 지정한 뒤, 각 데이터 열마다 완성된 PDF를 브라우저에서 생성하는 앱입니다.

화면	상태와 로직	저장과 파일	PDF 처리
React 19 TypeScript Vite	App 도메인 혹은 선택 혹은	Firebase Auth Firestore Storage	pdf-lib pdf.js fontkit

전체 연결 관계



별도의 백엔드 애플리케이션 서버는 없습니다. 브라우저가 Firebase에 직접 접속하고, PDF 연산도 브라우저에서 수행합니다.

사용자가 작업 화면에 들어오기까지

1 앱 실행 및 Google 로그인 확인



2 사용자 프로필에서 category 확인



3 같은 category의 워크스페이스 목록 로딩



4 워크스페이스 선택



5 표 데이터와 PDF 데이터를 동시에 로딩



6 실제 작업 화면 표시

여기서 가장 중요한 단위

현재 시스템의 실제 작업 단위는 워크스페이스입니다. 표, 이미지, PDF 원본, 출력 영역, 미세조정 설정이 모두 선택한 워크스페이스 경로 아래에서 함께 관리됩니다.

무엇이 어디에 저장되는가

작은 데이터와 큰 파일을 서로 다른 저장소에 나눠 보관합니다.

Cloud Firestore	Firebase Storage
워크스페이스 정보 행과 열 PDF 파일 메타데이터 PDF 출력 영역 열별 미세조정 폰트 메타데이터	원본 PDF 파일 셀 이미지 파일 업로드한 폰트 파일

워크스페이스 내부 데이터

```
categories / {category}
└─ workspaces / {workspaceId}
    ├── fieldRows - 항목 행
    ├── valueColumns - 값 열과 셀 데이터
    ├── pdfSlotRows - PDF 매핑 행
    ├── pdfAreas - 공통 출력 영역
    ├── columnPdfAdjusts - 열별 미세조정
    └─ settings - 폰트 설정
```

현재 원본 PDF의 위치

Storage: categories / {category} / workspaces / {workspaceId} / pdfs / {pdfRowId}

현재 원본 PDF는 독립 보관함의 파일이 아니라 PDF 매핑 행에 종속되어 있습니다.

표의 행과 열은 어떤 의미인가

행은 입력 항목의 종류, 열은 실제 출력 대상 한 건입니다.

항목	고객 A	고객 B
성명	홍길동	김영희
주소	서울특별시 ...	부산광역시 ...
계약일	2026-07-14	2026-07-15

PDF와 연결될 때

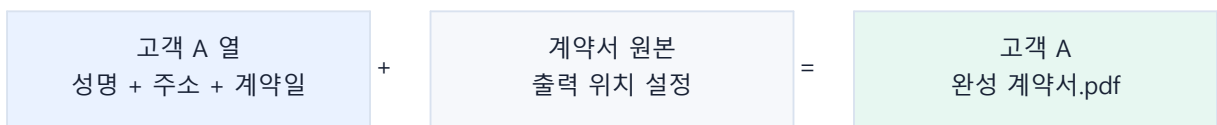


표 영역의 주요 기능

행과 열 추가 / 이름 변경 / 열 복제 / 다중 셀 복사·붙여넣기 / 셀 삭제와 실행 취소 / 이미지 업로드 / 자동 저장

원본 PDF와 출력 설정의 관계

1. PDF 매핑 행	2. 기준 영역	3. 열별 미세조정
어떤 원본 PDF를 사용할지 결정합니다. 현재 파일은 매핑 행과 1:1로 연결됩니다.	몇 페이지의 어느 위치에 어떤 표 항목을 출력할지 정합니다. 모든 열이 공유합니다.	특정 열에만 위치, 크기, 글자 설정을 덧씌웁니다. 필요한 경우에만 존재합니다.



다운로드 버튼을 누르면 내부에서 일어나는 일

1	Storage에서 원본 PDF를 Blob으로 가져옵니다.
2	pdf-lib가 원본을 PDFDocument로 엽니다.
3	한글 출력을 위한 폰트를 PDF에 포함합니다.
4	각 기준 영역에 현재 열의 텍스트 또는 이미지를 그립니다.
5	기준 영역과 열별 미세조정 값을 합쳐 최종 위치를 계산합니다.
6	완성된 PDF를 Uint8Array 바이트로 생성합니다.
7	브라우저에서 파일을 다운로드하고 임시 URL을 해제합니다.

결과 PDF는 Firebase Storage에 저장되지 않습니다. 현재도 브라우저 메모리에서 생성한 뒤 바로 다운로드하는 구조입니다.

관련 모듈

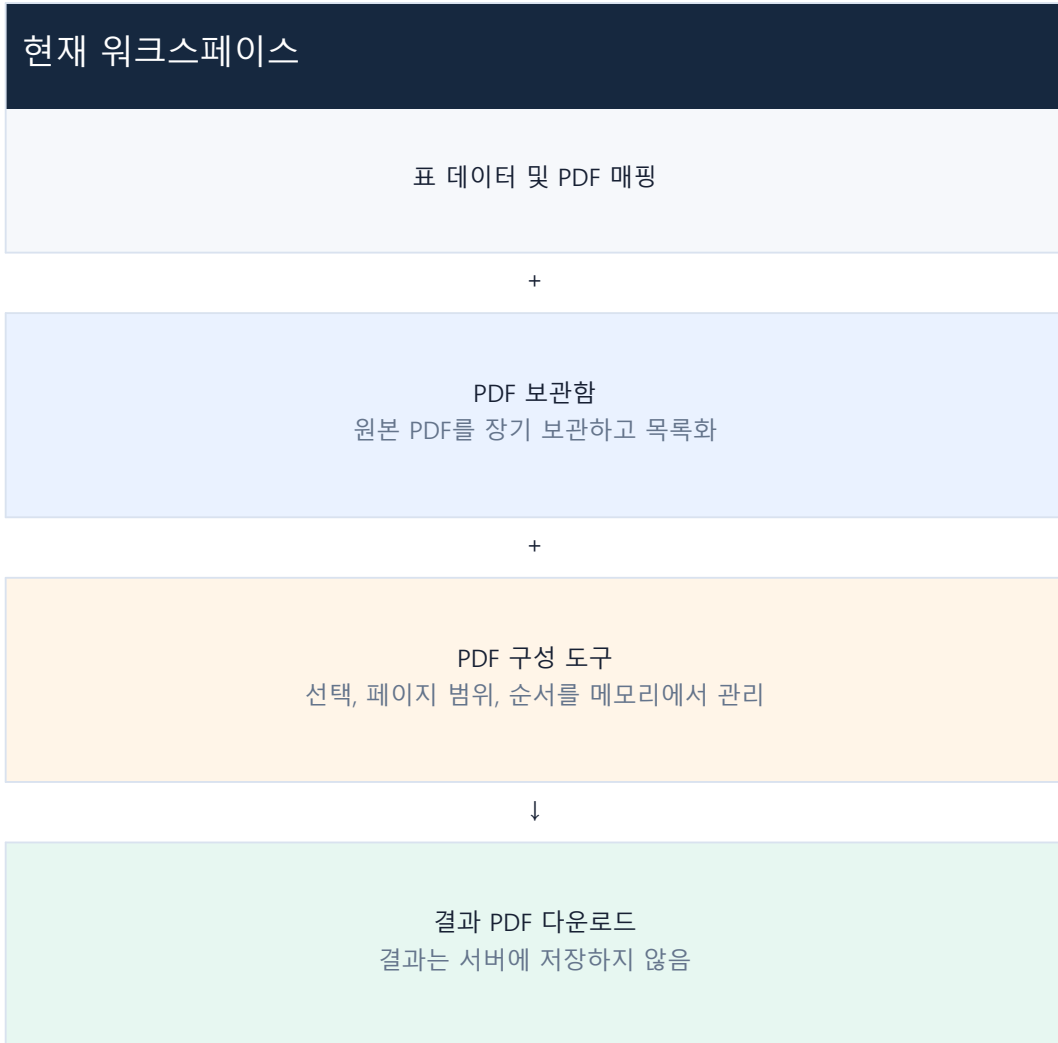
pdfExport.ts - 전체 PDF 생성과 병합
 geometry.ts - 화면 좌표를 PDF 좌표로 변환
 textRenderer.ts / imageRenderer.ts - 텍스트와 이미지 출력
 fonts.ts - 한글 폰트 처리

주요 파일을 역할별로 찾기

파일	역할
src/App.tsx	로그인, 워크스페이스, 표, PDF, 모달을 연결하는 전체 조정자
src/hooks/useSheetData.ts	행, 열, 셀 값, 이미지와 표 저장 로직
src/hooks/usePdfData.ts	PDF 행, 영역, 미세조정, 출력과 다운로드
src/hooks/useSheetSelection.ts	셀 범위 선택, 복사, 선택 상태
src/services/storage.ts	Firestore와 Storage를 모두 감싸는 repository
src/services/pdfExport.ts	PDF 렌더링, 병합, 다운로드
src/components/PdfSetupModal.tsx	PDF 출력 영역 편집과 결과 미리보기
src/types.ts	프로젝트 핵심 데이터 타입

PDF 보관함과 분리·병합 기능은 어디에 들어갈까

현재의 워크스페이스 단위는 유지하고, 그 안에 두 개의 독립 영역을 추가하는 방향이 자연스럽습니다.



기존 코드 중 재사용 가능한 부분

워크스페이스 경로 구조 / Firebase Storage 업로드·다운로드 / pdf-lib 병합 함수 / 브라우저 다운로드 / 진행 상태와 알림 UI

이 프로젝트를 이해할 때 기억할 6가지

- 1 워크스페이스가 모든 작업 데이터의 기본 단위입니다.
- 2 App.tsx가 화면과 여러 도메인 혹은 연결합니다.
- 3 Firestore에는 구조화된 데이터, Storage에는 실제 파일이 저장됩니다.
- 4 표의 한 열이 완성 PDF 한 세트의 데이터가 됩니다.
- 5 PDF 출력은 공통 기준 영역과 선택적인 열별 미세조정으로 구성됩니다.
- 6 PDF 결과 생성과 병합은 브라우저 메모리에서 처리됩니다.

다음 작업을 시작한다면 가장 작은 첫 단위는, 기존 PDF 매핑을 건드리지 않고 워크스페이스 안의 PDF 보관함 데이터 모델과 목록 읽기부터 정의하는 것입니다.